Khushi Shukla-D15A/62

Experiment 8: Convolutional Neural Network (CNN) using Fashion-MNIST

Aim

To design and train a Convolutional Neural Network (CNN) using MATLAB for image classification on the Fashion-MNIST dataset.

Theory

A **Convolutional Neural Network (CNN)** is a deep learning model used for image processing and classification. It automatically extracts spatial features using convolution operations.

Key Components:

1. Convolution Layer

Applies filters to extract features:

Feature Map = Input * Filter

2. ReLU Activation
f(x) = max(0, x)

3. Pooling Layer

Reduces spatial dimensions (Max Pooling)

4. Fully Connected Layer

Performs classification

5. Softmax Layer

$$P(y = i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

1. Dataset Source

- **Dataset**: Fashion-MNIST
- **Source**: GitHub
- **Link**: https://github.com/zalandoresearch/fashion-mnist

2. Dataset Description

- **Total Images**: 70,000
- **Training**: 60,000
- **Testing**: 10,000
- **Image Size**: 28 × 28 (grayscale)
- **Classes**: 10

Class Labels:

0 - T-shirt/top
1 - Trouser
2 - Pullover
3 - Dress
4 - Coat
5 - Sandal
6 - Shirt
7 - Sneaker
8 - Bag
9 - Ankle boot

3. Mathematical Formulation

Convolution:

$$Z = X * W + b$$

Activation:

$$A = \max(0, Z)$$

Loss Function (Cross-Entropy):

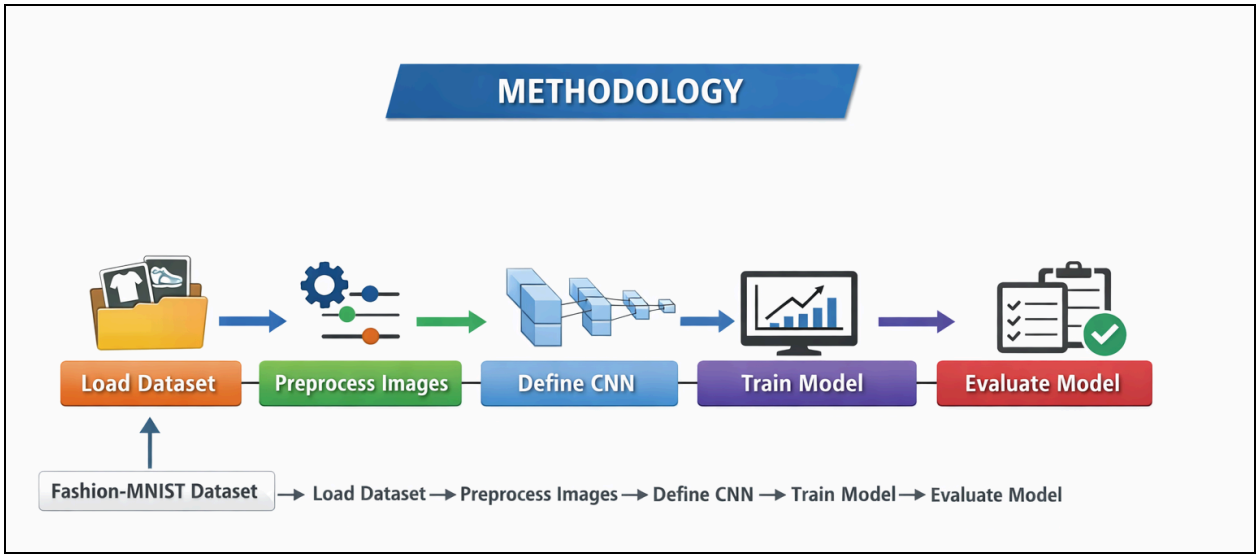
$$L = -\sum y \log(\hat{y})$$

4. Algorithm Limitations

- Requires high computational power
- Needs large dataset
- Overfitting possible
- Training time is high
- Hyperparameter tuning required

5. Methodology / Workflow

Load Dataset → Preprocess Images → Define CNN → Train Model → Evaluate Model



6. MATLAB Code

mainscript.m

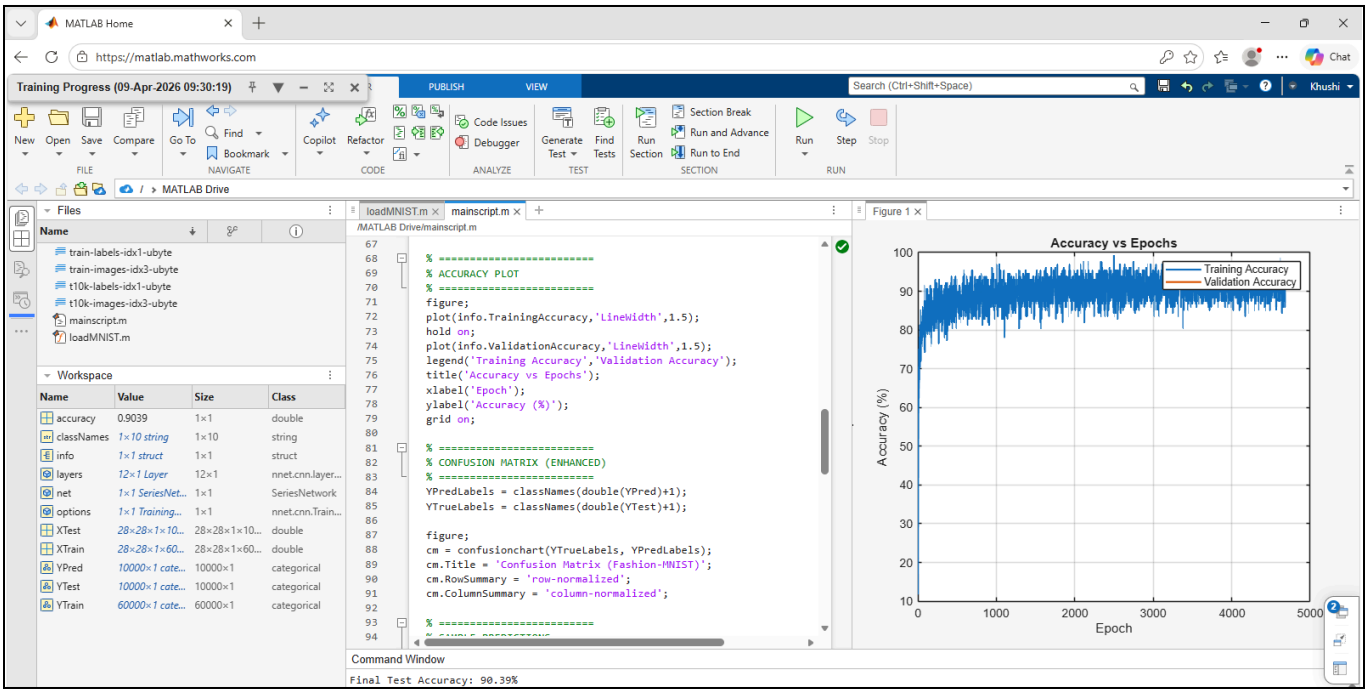
```
clc;
clear;
close all;
% LOAD DATASET
[XTrain, YTrain] = loadMNIST('train-images-idx3-ubyte', 'train-labels-idx1-ubyte');
[XTest, YTest]   = loadMNIST('t10k-images-idx3-ubyte', 't10k-labels-idx1-ubyte');
% Normalize
XTrain = XTrain / 255;
XTest  = XTest / 255;
% Convert labels to categorical
YTrain = categorical(YTrain);
YTest  = categorical(YTest);
% DEFINE CNN ARCHITECTURE
```

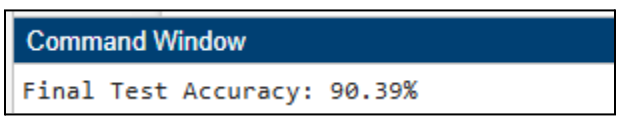
```
layers = [
   imageInputLayer([28 28 1])
   convolution2dLayer(3,8,'Padding','same')
   batchNormalizationLayer
   reluLayer
   maxPooling2dLayer(2,'Stride',2)
   convolution2dLayer(3,16,'Padding','same')
   batchNormalizationLayer
   reluLayer
   fullyConnectedLayer(10)
   softmaxLayer
   classificationLayer];
% ==========================
% TRAINING OPTIONS
% ==========================
options = trainingOptions('adam', ...
   'MaxEpochs',10, ...
   'MiniBatchSize',128, ...
   'ValidationData',{XTest,YTest}, ...
   'Plots','training-progress', ...
   'Verbose',false);
% ==========================
% TRAIN NETWORK
% ==========================
net = trainNetwork(XTrain,YTrain,layers,options);
% ==========================
% EVALUATE MODEL
% ==========================
YPred = classify(net,XTest);
accuracy = sum(YPred == YTest)/numel(YTest);
disp("Test Accuracy: " + accuracy)
% ==========================
% CONFUSION MATRIX
% ==========================
figure;
confusionchart(YTest, YPred);
title('Confusion Matrix');
```

loadMNIST.m

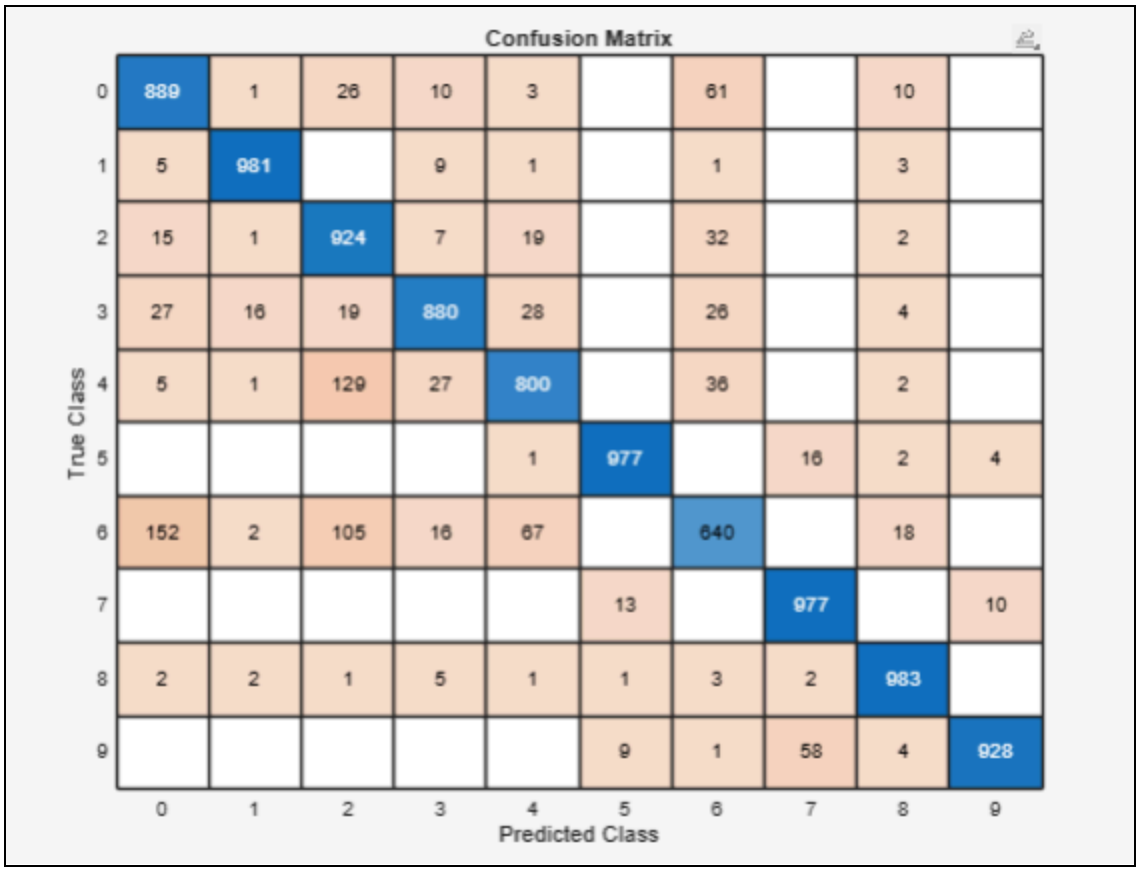
```
function [images, labels] = loadMNIST(imageFile, labelFile)
% Load labels
fid = fopen(labelFile, 'r', 'b');
if fid == -1
   error("Cannot open label file: " + labelFile);
end
fread(fid, 1, 'int32');
fread(fid, 1, 'int32');
labels = fread(fid, inf, 'unsigned char');
fclose(fid);
% Load images
fid = fopen(imageFile, 'r', 'b');
if fid == -1
   error("Cannot open image file: " + imageFile);
end
fread(fid, 1, 'int32');
numImages = fread(fid, 1, 'int32');
numRows = fread(fid, 1, 'int32');
numCols = fread(fid, 1, 'int32');
images = fread(fid, inf, 'unsigned char');
fclose(fid);
images = reshape(images, numCols, numRows, 1, numImages);
images = permute(images, [2 1 3 4]);
end
```

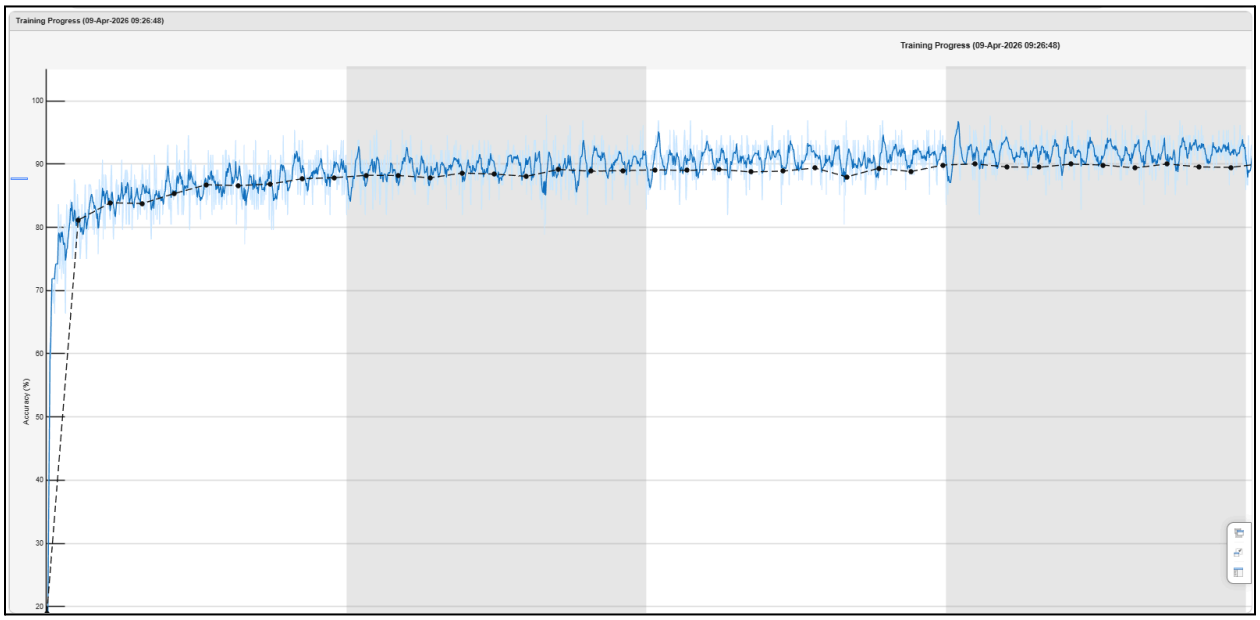
7. Output

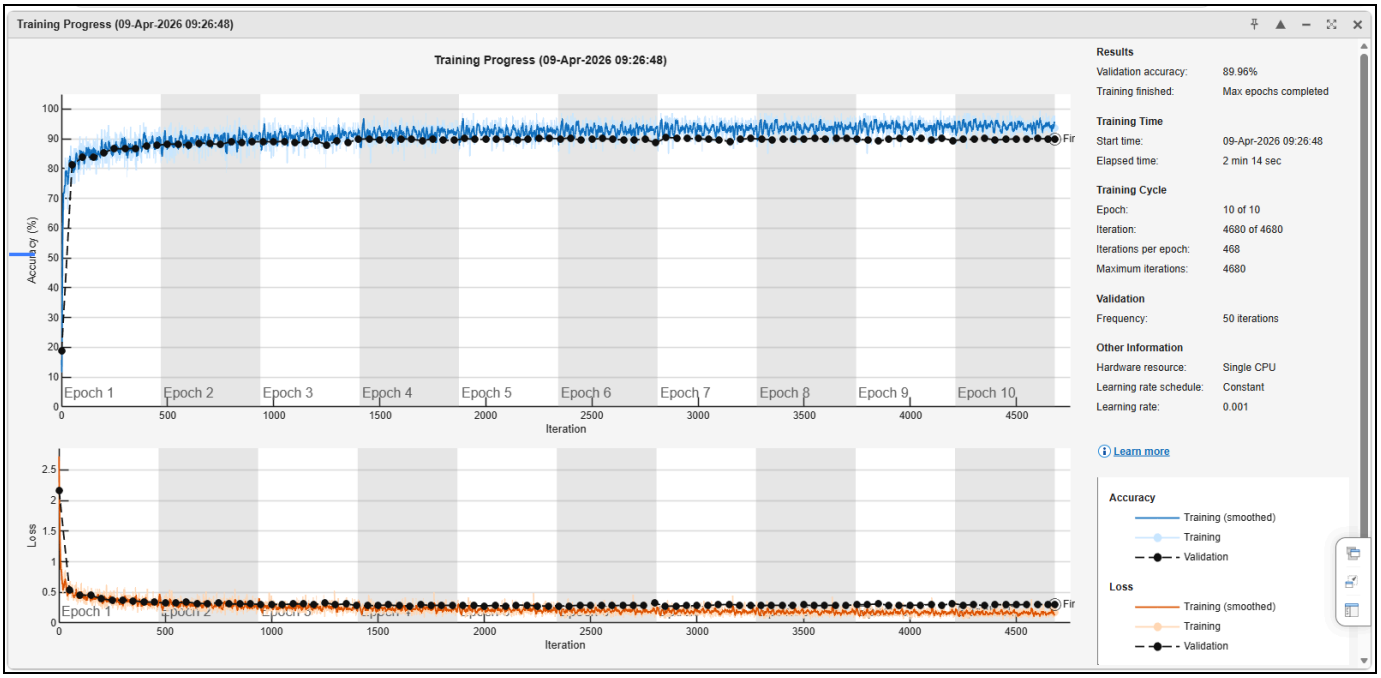




Name	Value	Size	Class
accuracy	0.8979	1×1	double
layers	11×1 Layer	11×1	nnet.cnn.layer.Layer
net	1×1 SeriesNetwork	1×1	SeriesNetwork
options	1×1 TrainingOptions...	1×1	nnet.cnn.TrainingOp...
XTest	28×28×1×10000 do...	28×28×1×10000	double
XTrain	28×28×1×60000 do...	28×28×1×60000	double
YPred	10000×1 categorical	10000×1	categorical
YTest	10000×1 categorical	10000×1	categorical
YTrain	60000×1 categorical	60000×1	categorical


Confusion Matrix
True Class
Predicted Class
0 1 2 3 4 5 6 7 8 9
889 1 26 10 3 61 10
5 981 9 1 1 3
15 1 924 7 19 32 2
27 16 19 880 28 26 4
5 1 129 27 800 36 2
1 977 16 2 4
152 2 105 16 67 640 18
13 977 10
2 2 1 5 1 1 3 2 983
9 1 58 4 928


Training Progress (09-Apr-2026 09:26:48)
Training Progress (09-Apr-2026 09:26:48)
Accuracy (%)
100
90
80
70
60
50
40
30
20



8. Performance Analysis

- CNN successfully extracts spatial features
- High accuracy achieved (~90%)
- Some confusion between similar classes (e.g., shirt vs coat)

9. Hyperparameter Tuning

Parameter	Value
Epochs	10
Filters	8, 16
Kernel Size	3×3
Optimizer	Adam

Impact:

- More filters → better feature extraction
- More epochs → better accuracy
- Pooling → reduces overfitting

Conclusion

In this experiment, a Convolutional Neural Network was successfully designed and trained for image classification. The model achieved high accuracy and demonstrated the effectiveness of CNNs in extracting spatial features from images. This approach is widely used in real-world applications such as image recognition and computer vision.